Measuring the architectural maintainability of iOS code generated by AI coding agents

October 2026

Agents write the code now

You describe a task in text — the agent edits the project.

But green build says nothing about structure.

```
final class EventsPresenter {
    let db = Database.shared // a feature now reaches into storage
}
```

Nothing fails. No test breaks. No tool warns.

Goal

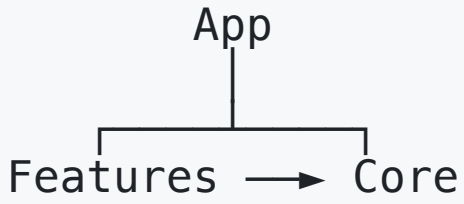
1. Build an instrument to **measure maintainability** of agent-written iOS code.
2. Use it to test one idea:
   if the agent is given **a set of architecture rules**, does its code get structurally better?

What others write about this

Metrics and architectural smells over a dependency graph

Dependecy graph

Every architecture metric starts from dependency graph.



A **component** is a feature or a some part of it (not a single file!).

Propagation cost

How far a change spreads

$$PC = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} V_{ij}$$

N — number of components

V_{ij} — 1 if component i reaches component j (directly or via a chain), else 0

MacCormack, Rusnak & Baldwin

Instability

How locked-in a part is

$$I(C) = \frac{C_e}{C_a + C_e}$$

C_a — how many components depend on C
C_e — how many components C depends on

$I = 0$ — stable, everyone relies on it, expensive to change
$I = 1$ — relies on everyone, nobody relies on it, free to change

Martin; validated by Hyrynsalmi & Leppänen

Four architectural smells

These are named patterns of the numbers above

Azadi, Fontana & Taibi

Fontana et al.

Mo et al.

Sas et al.

Smell 1: Cyclic Dependency

```
final class PlaybackService {
    let library: LibraryService   // depends on LibraryService
}

final class LibraryService {
    let player: PlaybackService   // depends on PlaybackService
}
```

Smell 2: God Component

```
Core/Common/          8,716 LOC   74 files
   ├── Configs
   ├── Models
   ├── Factories
   ├── Services
   └── ...
```

Smell 3: Hub-Like Dependency

```
final class APIService { // depends on 7, is depended on by 5
    let auth:      AuthService
    let cache:     CacheService
    let analytics: AnalyticsService
    // ...
}
final class OperationsService { let api: APIService }
final class TranchesService   { let api: APIService }
// ...
```

Both C_a (incoming) and C_e (outgoing) are high

Smell 4: Unstable Dependency

```
// Core/Playback — stable, 8 components depend on it
final class PlaybackService {
    func play(_ item: ShortsItem) { ... }
    //                ^ this type belongs to another feature
}
```

A **stable** component (low I) depends on a **non-stable** one (high I).

The approach

The profile

Aspect	Metric	Purpose
Build	build, tests, behaviour	— (a gate filter)
Size balance	LOC, Gini	GC
Dependencies coupling	C_a, C_e, cycles, I, PC_w	CD, HLD, UD
Rules	forbidden edges	— (explicit rules)

Size balance — God Component

$$\frac{\max_i \mathrm{LOC}_i}{\mathrm{median}_i \mathrm{LOC}_i} \qquad G = \frac{\sum_i \sum_j |x_i - x_j|}{2n^2 \bar{x}}$$

x_i — size of part i, n — number of parts, $\bar{x}$ — average size

G — Gini:

- 0 = all equal
- 1 = one part holds everything

Dependencies coupling — CD, HLD, UD

- **cycles** (count + largest) — **Cyclic Dependency**
- high C_a **and** high C_e — **Hub-Like Dependency**
- stable part depending on a volatile one — **Unstable Dependency**
- PC_w — how far a change can now reach

Rules

Rules written **before** the run, checked on the same graph:

> *Core must not reference Features*
>
> *Features must not reference each other directly*

Wong et al., Terra & Valente

Collecting the metrics: tools

Level	Source	Gives
1	swift-code-metrics	LOC, classes, protocols
2	my **AST** analyzer (SwiftSyntax)	the dependency edges

Why a custom AST analyzer?

```
// Features/Player/PlayerViewModel.swift — no import needed
final class PlayerViewModel {
    let playback = PlaybackService()
    //             ^ declared in Core/Playback
}
```

Experiment flow

The experiment

Same project, same starting commit, same task text.

A	agent alone
B	agent + rule set

3–5 repetitions per condition — the agent is non-deterministic.

The rule set

1. **The agent is given written architecture rules.**
   Writing such rules formally is not new.

   Terra & Valente (DCL), Ribeiro et al., Wong et al.

2. **The same rules are also the ruler.**
   It separates "the agent did not know" from "the agent broke it anyway"

The data

- At least 30 tasks on different large open-source iOS projects.
- Each task comes from the project's **own history** — a real issue or merged pull request.
- A human reference solution exists, and the task was not invented to fit the result.

What is new

Platform — nothing validated exists for Swift, I will use a known approaches and measure what the port costs in accuracy

Profile — a set of metrics that are noisy

Rules used twice — as the agent's input and as the measuring stick

Thank you for attention